SIMON DICE POR VOZ

Documentacion Tecnica Completa

Proyecto Universitario - Sistemas Inteligentes

Hardware: ESP32-S3-N16R8 | Python + Whisper | Next.js 14

Alumnos: Leonardo Alonso Aldana y Jennifer Rubi Hernández Carrillo

Fecha: Abril 2026

Indice

- 1. Objetivo del proyecto
- 2. Descripcion general
- 3. Hardware — componentes fisicos y conexiones
- 4. Arquitectura del sistema — los tres modos
- 5. Diagramas de arquitectura (ASCII)
- 6. Modos de operacion
- 7. Maquina de estados del juego
- 8. Protocolo de comunicacion Serial
- 9. Protocolo WebSocket (Servidor -> Panel)
- 10. Pipeline de audio y reconocimiento de voz
- 11. Stack tecnologico completo
- 12. Librerias Arduino — justificacion detallada
- 13. Librerias Python — justificacion detallada
- 14. Librerias Web Panel — justificacion detallada
- 15. Estructura de carpetas del proyecto
- 16. Descripcion detallada de cada archivo
- 17. Firmware ESP32
- 18. Servidor Python
- 19. Panel Web — Next.js
- 20. Vocabulario y validador de comandos
- 21. Narradora TTS — frases y sonidos
- 22. Pines GPIO verificados
- 23. Hilos concurrentes del servidor
- 24. Display OLED — mensajes por estado
- 25. Setup e instalacion
- 26. Riesgos y decisiones de diseno

1. Objetivo del proyecto

Construir un juego Simon Dice controlado completamente por voz en espanol que corra sobre hardware real (ESP32-S3) y sea accesible desde un panel web. El jugador nunca toca un boton para jugar: habla, el sistema escucha, reconoce el color dicho y evalua si la secuencia es correcta. El sistema funciona completamente sin internet para el reconocimiento de voz (Whisper local).

Objetivos especificos

#	Objetivo	Estado
1	Capturar audio desde el ESP32 y transmitirlo a la PC	Verificado
2	Transcribir voz en espanol con Whisper local (sin internet)	Verificado
3	Implementar la logica del juego Simon Dice completa	Completado
4	Mostrar el estado del juego en tiempo real en un panel web	Completado
5	Reproducir tonos y feedback de audio desde el speaker del kit	Completado
6	Funcionar sin depender de servicios externos de nube	Completado
7	Soporte multi-color: reconocer varios colores en un solo audio	Completado
8	Fallback WASM: Whisper en el browser sin Python instalado	Completado

2. Descripcion general

Simon Dice por Voz es un sistema distribuido de tres capas que trabajan juntas en tiempo real. El jugador dice los colores en voz alta; el sistema los transcribe con inteligencia artificial local (Whisper), valida la secuencia y responde con sonidos y animaciones visuales en un panel web.

Capas del sistema

Capa	Componente	Funcion
1 — Hardware	ESP32-S3 + MAX4466 + MAX98357A + OLED + RGB	Captura audio | Reproduce tonos | Muestra estado en OLED | LED RGB de color
2 — Servidor	Python + Whisper + WebSocket + TTS	Motor del juego | Reconocimiento de voz | Narrador TTS | Bridge Serial
3 — Panel Web	Next.js 14 + TypeScript (Chrome/Edge)	Dashboard tiempo real | LEDs virtuales | Log | Score | Countdown

Flujo de una ronda completa

1. Jugador dice "EMPIEZA" o presiona boton
2. Python genera secuencia aleatoria -> envia LEDs al panel y tonos al ESP32
3. ESP32 muestra cada color (RGB + OLED) y toca el tono en el speaker MAX98357A
4. Panel web ilumina el LEDPanel correspondiente en tiempo real via WebSocket
5. ESP32 entra en estado LISTENING — panel muestra cuenta regresiva 15s
6. Jugador presiona ESPACIO (o boton SW1 fisico) y habla el color
7. Audio -> Python -> pipeline DSP (9 capas) -> Whisper -> comando
8. Python evalua: coincide con el color esperado?

    - SI -> CORRECT -> tono de acierto -> siguiente color o LEVEL_UP
    - NO -> WRONG -> tono de error -> GAME OVER

9. Narradora TTS (edge-tts DaliaNeural) verbaliza el resultado

3. Hardware — componentes fisicos y conexiones

Kit principal: OKYSTAR PCB 56483A_Y85_250412 (ESP32-S3-N16R8)

El kit MRD085A integra en una PCB compacta (50mm x 40mm) el ESP32-S3, pantalla OLED, botones de volumen y el LED RGB. El microfono MAX4466 y el amplificador MAX98357A son modulos externos conectados por jumpers en el protoboard.

Componente	Modelo	Funcion en el juego
MCU	ESP32-S3-N16R8	240MHz dual-core | 16MB Flash | 8MB PSRAM OPI | aceleracion ESP-NN
Microfono	MAX4466 (analogico)	Captura voz del jugador via ADC GPIO4 a 8kHz
Amplificador	MAX98357A (I2S digital)	Amplifica y reproduce tonos de juego hasta 3W en clase D
Speaker	Bocina pasiva 4-8 Ohm	Salida de audio — tonos de colores, acierto y error
Pantalla	OLED SSD1306 0.91" (128x32px)	Estado del juego, nivel, puntuacion, instrucciones
LED	WS2812B RGB NeoPixel (GPIO48)	Indica el color activo del juego con animacion
Boton SW1	GPIO0 (BOOT)	PTT fisico principal — presionar para hablar al juego
Boton SW2	GPIO35	PTT alternativo / funcion de pausa
Interfaz	USB-C	Alimentacion + puerto Serial a 921600 baud

Conexiones del MAX98357A al protoboard

El MAX98357A es un amplificador de clase D digital. Su salida es DIFERENCIAL — OUT+ y OUT- NUNCA deben conectarse a GND, solo entre si a los terminales de la bocina.

Pin MAX98357A	Conectar a	Descripcion
LRC (WS)	GPIO 5 del ESP32	I2S Word Select — sincroniza canal izq/der
BCLK	GPIO 6 del ESP32	I2S Bit Clock — reloj de datos a 16000Hz * 16bit * 2 = 512kHz
DIN	GPIO 7 del ESP32	I2S Data In — datos de audio digital
GAIN	Sin conectar (flotando)	Flotando = ganancia 15dB = maxima potencia

SD (Shutdown)	Sin conectar (flotando)	Flotando = amplificador habilitado
GND	Rail GND del protoboard	Masa comun con el ESP32
VIN	5V o 3.3V del ESP32	5V = mas potencia (3W) | 3.3V = menos potencia (~1W)
OUT+	Terminal (+) de la bocina	Salida diferencial positiva — NUNCA a GND
OUT-	Terminal (-) de la bocina	Salida diferencial negativa — NUNCA a GND

Conexiones del MAX4466 al protoboard

Pin MAX4466	Conectar a	Descripcion
VCC	3.3V del ESP32	Alimentacion del modulo
GND	GND del ESP32	Masa comun
OUT	GPIO4 del ESP32	Salida de audio analogico hacia el ADC del ESP32

Bus de comunicacion del hardware

```
ESP32-S3

   |

   |-- I2S0 (GPIO12/13/11) -----> INMP441 (microfono digital — no usado
aun)

   |-- I2S1 (GPIO6/5/7)   -----> MAX98357A (amplificador speaker)

   |-- I2C  (GPIO10/11)   -----> OLED SSD1306 128x32

   |-- ADC  (GPIO4)       -----> MAX4466 (microfono analogico — en uso)

   |-- GPIO (GPIO48)      -----> WS2812B RGB NeoPixel

   |-- GPIO (GPIO0)       -----> SW1 Boton BOOT/PTT (INPUT_PULLUP, activo
LOW)

   |-- GPIO (GPIO35)      -----> SW2 Boton SW2 (INPUT_PULLUP)

   |-- USB Serial (921600 baud) -> PC / Chrome / Edge
```

4. Arquitectura del sistema — los tres modos

El sistema tiene tres formas de funcionar. Todos los modos comparten el mismo Web Panel. La diferencia esta en donde corre el juego y quien transcribe la voz.

	Modo A — Simulador	Modo B — ESP32 + Python	Modo C — ESP32 solo
Necesita hardware ESP32?	No	Si	Si
Necesita Python?	Si (obligatorio)	Si (servidor_voz)	No
Donde corre el juego?	Python (PC)	Firmware ESP32	Firmware ESP32
Quien transcribe la voz?	Whisper Python	Whisper Python	Whisper WASM en browser
Que microfono usa?	Mic del sistema (PC)	MAX4466 del kit o mic PC	MAX4466 del kit o mic browser
Hay OLED y speaker fisico?	No	Si	Si
Se puede desplegar en Vercel?	No	No	Si
Para que sirve?	Pruebas sin hardware	Entrega con mayor precision	Entrega autonoma final

5. Diagramas de arquitectura

Los diagramas originales del proyecto estan escritos en lenguaje Mermaid (renderizables en GitHub). A continuacion se representan en formato texto/ASCII para este documento.

Diagrama 1 — Los tres modos de operacion

```
+---------------------------+  +---------------------------+  +-----------
----------------+

|   MODO A — Simulador PC   |  | MODO B — ESP32 + servidor |  |  MODO C —
ESP32 solo      |

+---------------------------+  +---------------------------+  +-----------
----------------+

| python main.py            |  | Kit MRD085A               |  | Kit
MRD085A               |

| juego_sim.py              |  | ESP32-S3-N16R8            |  | ESP32-S3-
N16R8            |

| WebSocket :8765           |  | INMP441 + MAX98357A       |  | Sin Python
adicional      |

| Whisper Python            |  | OLED + SW1/SW2            |  | Sin
instalacion           |

| TTS edge-tts / SAPI       |  | servidor_voz.py           |  |
|

| LEDs virtuales ANSI       |  | Whisper Python :8766      |  |
|

+----------+----------------+  +----------+----------------+  +----------
+----------------+

           |                             |                             |

           | WebSocket JSON :8765        | USB Serial 921600 baud      |
USB Serial 921600 baud

           |                             |                             |

           v                             v                             v

+------------------------------------------------------------------------+

|                    WEB PANEL — Chrome o Edge                           |

|  useWebSocket.ts  |  useWebSerial.ts  |  useWhisperWASM.ts             |

|  GameStatus  LEDPanel  SequenceDisplay  LogConsole  ScoreBoard         |

+------------------------------------------------------------------------+
```

Diagrama 2 — Flujo de PTT con boton fisico (Modo B)

```
Jugador presiona SW1
    |
    v
ESP32: GPIO flanco de bajada
    | pausarTimeout()  — timer de 15s se congela
    | audioCapturaIniciar()  — INMP441 empieza a grabar en PSRAM
    | Serial -> BTN_INICIO -> Browser muestra 'Grabando...'
    |
    | [mientras el boton esta presionado]
    |   audioCapturaLoop() — lee bloques I2S y guarda en PSRAM
    |
Jugador suelta SW1
    |
    v
ESP32: GPIO flanco de subida
    | audioCapturaPararYEnviar()
    | Serial -> AUDIO:START:64000
    | Serial -> [lineas en base64]
    | Serial -> AUDIO:END
    |
Browser: decodifica base64 -> Float32Array
    | WebSocket -> servidor_voz.py -> Whisper transcribe
    | Whisper: 'rojo' -> normalizar -> ROJO
    | WebSocket -> Browser -> Serial -> ROJO -> ESP32
    |
ESP32: procesarComando(CMD_ROJO)
    | reanudarTimeout()  — timer sigue
    | Serial -> RESULT:CORRECT / RESULT:WRONG
```

Diagrama 3 — Pipeline completo de audio

```
MAX4466 (GPIO4, ADC)

    | 8kHz, int16, oversample x4 (firmware)

    v

HPF biquad 80Hz (firmware C++)

    | elimina DC offset del MAX4466

    v

LPF biquad 4000Hz (firmware C++)

    | limita a banda de voz, elimina aliasing

    v

USB Serial -> PCM int16 bytes -> Python

    v

HPF scipy.signal 80Hz (filtfilt, fase cero)

    v

LPF scipy.signal 3400Hz (filtfilt, fase cero)

    v

noisereduce (sustraccion espectral 55%)

    v

Normalizacion al 90% del pico (numpy)

    v

Resample 8kHz -> 16kHz (scipy)

    v

Whisper small — float32 PCM 16kHz -> texto espanol

    v

Validador: quitar acentos + fuzzy match -> AZUL / ROJO / etc.
```

Diagrama 4 — Protocolo WebSocket (Servidor -> Panel)

```
Servidor Python                          Web Panel (Next.js)

    |                                        |

    |--- ready {whisperDisponible:true} ---> GameStatus: "Listo"
```

```
    |--- state {estado:"SHOWING"}      ---> GameStatus actualiza badge

    |--- led {color:"ROJO"}            ---> LEDPanel enciende ROJO

    |--- sequence {secuencia:[...]}    ---> SequenceDisplay muestra items

    |--- expected {esperado:"ROJO"}    ---> GameStatus muestra "Esp: ROJO"

    |--- tts {activo:true}             ---> countdown pausa, spacebar
bloqueado

    |--- tts {activo:false}            ---> countdown reanuda

    |--- voz {texto:"azul",cmd:"AZUL"} ---> GameStatus muestra
transcripcion

    |--- result {resultado:"CORRECT"}  ---> badge verde, reset flags PTT

    |--- level {nivel:2}               ---> ScoreBoard actualiza nivel

    |--- score {puntuacion:10}         ---> ScoreBoard actualiza puntos

    |--- gameover                      ---> pantalla Game Over

    |--- log {mensaje:"..."}           ---> LogConsole append
```

6. Modos de operacion

Modo	Reconocimiento de voz	PTT	Python necesario	Calidad ASR
Simulador PC	Whisper Python local (small)	Barra espaciadora	Si	Alta
ESP32 + servidor_voz	Whisper Python local (small)	Boton SW1 o espacio	Si	Alta
ESP32 + WASM	Whisper WASM en browser (q4)	Boton SW1 o espacio	No	Media

Seleccion automatica del modo Whisper

Al conectar el browser, el servidor Python envia un mensaje de tipo 'ready' con el campo 'whisperDisponible'. Si es true, el browser usa PTT local (Python graba el microfono del PC). Si es false, el browser descarga automaticamente Whisper WASM (~125MB, cuantizado INT4, se cachea en IndexedDB del browser para usos posteriores).

7. Maquina de estados del juego

Estado	Descripcion	Transicion de salida
IDLE	Esperando que el jugador diga START o presione boton	START -> SHOWING_SEQUENCE
SHOWING_SEQUENCE	Muestra los colores uno por uno (LED + tono + OLED)	Secuencia completa -> LISTENING
LISTENING	Turno del jugador — espera voz (timeout 15s, pausado durante TTS/Whisper)	PTT_FIN -> EVALUATING | timeout -> GAME_OVER | PAUSA -> PAUSA
PAUSA	Juego detenido temporalmente	START / PAUSA -> LISTENING
EVALUATING	Procesa la respuesta recibida	correcto -> CORRECT | incorrecto -> WRONG
CORRECT	Respuesta correcta — tono de acierto — 0.6s delay	fin secuencia -> LEVEL_UP | no fin -> LISTENING
LEVEL_UP	Secuencia crece en 1 — fanfarria — nivel actualizado	SHOWING_SEQUENCE (nivel++)
WRONG	Respuesta incorrecta — tono de error — 0.8s delay	GAME_OVER
GAME_OVER	Fin del juego — muestra puntuacion final en OLED	START / REINICIAR -> IDLE

Parametros configurables del juego (vocabulario.h)

Parametro	Valor por defecto	Descripcion
TIMEOUT_RESPUESTA_MS	15 000 ms	Tiempo maximo para responder. Se pausa durante TTS y Whisper.
DURACION_LED_MS	800 ms	Tiempo que brilla cada color de la secuencia
PAUSA_ENTRE_LEDS_MS	300 ms	Silencio entre colores al mostrar la secuencia
NIVEL_INICIAL	1	Longitud inicial de la secuencia
MAX_NIVEL	20	Nivel maximo alcanzable

8. Protocolo de comunicacion Serial

Formato: texto plano, una linea por mensaje, terminado en \n. Baud rate: 921600. Esto permite leer el protocolo directamente en el Serial Monitor del Arduino IDE.

ESP32 -> PC / Browser

Mensaje	Descripcion
READY	Sistema inicializado y listo
STATE:IDLE	Esperando inicio
STATE:SHOWING	Mostrando secuencia de colores
STATE:LISTENING	Turno del jugador
STATE:EVALUATING	Procesando respuesta
STATE:GAMEOVER	Fin del juego
LED:ROJO / VERDE / AZUL / AMARILLO	LED del color correspondiente activo
LED:OFF	LEDs apagados
RESULT:CORRECT	Respuesta correcta
RESULT:WRONG	Respuesta incorrecta
RESULT:TIMEOUT	No hablo a tiempo
SEQUENCE:ROJO,AZUL,VERDE	Secuencia completa del nivel actual
EXPECTED:VERDE	Color que debe decir en este turno
LEVEL:3	Nivel actual actualizado
SCORE:30	Puntuacion actual actualizada
GAMEOVER	Fin del juego
BTN_INICIO	Boton SW1/SW2 presionado — inicio de captura de audio
AUDIO:START:64000	Inicio de bloque de audio (numero de bytes a recibir)
AUDIO:END	Fin del bloque de audio base64

PC / Browser -> ESP32

Mensaje	Descripcion
ROJO / VERDE / AZUL / AMARILLO\n	Color reconocido por Whisper
START\n	Iniciar juego (equivale a decir EMPIEZA)
STOP\n	Detener juego
PAUSA\n	Pausar el turno actual
REPITE\n	Repetir la secuencia de este nivel
REINICIAR\n	Reiniciar desde nivel 1
DESCONOCIDO\n	No se reconocio la voz o error
PTT_INICIO\n	Jugador empieza a hablar — pausa el timer
PTT_FIN\n	Jugador termino de hablar — reanuda el timer

9. Protocolo WebSocket (Servidor -> Panel)

Todos los mensajes WebSocket son JSON. El campo 'tipo' es el discriminador. El servidor corre en ws://localhost:8766 (modo B) o ws://localhost:8765 (modo A simulador).

tipo	Campos adicionales	Efecto en el panel
ready	whisperDisponible: bool, tiempoTimeout: int	Muestra estado 'Listo', configura modo Whisper
state	estado: string (IDLE|SHOWING|LISTENING|...)	Actualiza badge de estado del juego
led	color: string | null	Enciende el LED virtual del color en LEDPanel
sequence	secuencia: string[]	Muestra la secuencia de este nivel en SequenceDisplay
expected	esperado: string	Muestra el color esperado en GameStatus
voz	texto: string, comando: string	Muestra transcripcion Whisper, limpia flags PTT
result	resultado: CORRECT|WRONG|TIMEOUT	Badge verde/rojo, reset flags PTT
level	nivel: int	Actualiza numero de nivel en ScoreBoard
score	puntuacion: int	Actualiza puntuacion en ScoreBoard
gameover	(sin campos)	Muestra pantalla Game Over
tts	activo: bool	true: pausa countdown y bloquea spacebar | false: reanuda
log	mensaje: string	Append al LogConsole en tiempo real

10. Pipeline de audio y reconocimiento de voz

El audio capturado por el MAX4466 (analogico, 8kHz) pasa por 9 capas de procesamiento antes de llegar a Whisper. Esto es necesario porque el MAX4466 es un microfono analogico con mas ruido que un microfono I2S digital (INMP441), y Whisper requiere 16kHz.

Capa	Tecnica	Descripcion
0	ADC oversample x4 (firmware C++)	Lee GPIO4 cuatro veces y promedia. Reduce ruido ADC ~6dB. int16 a 8kHz.
1	HPF biquad 80Hz (firmware C++)	Elimina DC offset del MAX4466 y vibraciones de baja frecuencia. Coeficientes precalculados.
2	LPF biquad 4000Hz (firmware C++)	Limita a banda de voz. Elimina aliasing antes del resample. Coeficientes precalculados.
3	HPF scipy.signal 80Hz (Python)	Segunda pasada con filtfilt (fase cero). Sin distorsion temporal de la senal de voz.
4	LPF scipy.signal 3400Hz (Python)	Segunda pasada con filtfilt. Banda de voz telefonica final.
5	noisereduce (Python)	Sustraccion espectral del 55% del ruido ADC estacionario. Estima perfil de ruido del primer 15% del audio.
6	Normalizacion 90% pico (numpy)	Maximiza SNR sin saturar. Pico maximo = 0.9 de amplitud.
7	Resample 8kHz -> 16kHz (scipy)	Whisper requiere exactamente 16kHz. scipy.signal.resample con interpolacion.
8	Whisper small (openai-whisper)	float32 PCM 16kHz -> texto en espanol. language=es, temp=0.0, beam=5.
9	Validador texto -> comando	Quita acentos + regex + fuzzy match difflib -> AZUL, ROJO, VERDE, AMARILLO, etc.

Modelos Whisper disponibles

Modelo	Tamano	Velocidad CPU	Calidad	Uso en el proyecto
tiny	~39 MB	~0.5s	Baja	Fallback rapido (config_voz.py)

base	~74 MB	~1-2s	Media	Pruebas de desarrollo
small	~244 MB	~3-8s	Alta	Produccion (configurado por defecto)
WASM small INT4	~125 MB	~1-3s	Media	Fallback browser (Modo C)

Parametros de Whisper configurados

```
modelo.transcribe(
    audio,
    language='es',          # forzar espanol
    temperature=0.0,        # determinista, sin aleatoriedad
    fp16=False,             # CPU — fp16 es solo para GPU
    beam_size=5,            # busqueda de haz — mayor calidad
    no_speech_threshold=0.50,  # umbral para detectar silencio
)
```

11. Stack tecnologico completo

Resumen por capa

Capa	Tecnologia	Por que se usa
Hardware MCU	ESP32-S3-N16R8	16MB Flash + 8MB PSRAM OPI. Aceleracion ESP-NN para TFLite en Fase 2.
Hardware mic	MAX4466 (analogico)	Bajo costo. ADC directo. Suficiente con pipeline DSP de 9 capas.
Hardware speaker	MAX98357A (I2S digital)	Amplificador clase D integrado. 3W. Digital: sin ruido de cables.
Firmware	Arduino Framework / ESP-IDF	Abstracciones para I2C, I2S, NeoPixel sin drivers ESP-IDF manuales.
ASR voz	openai-whisper small	100% local. Sin internet. Espanol nativo. 15 palabras de vocabulario.
DSP audio	scipy + numpy	Pipeline profesional. filtfilt (fase cero) preserva forma temporal de voz.
Reduccion ruido	noisereduce	Sustraccion espectral. Mejora reconocimiento del MAX4466 ruidoso.
TTS narrador	edge-tts (DaliaNeural)	Voz neural es-MX. Calidad muy alta. Fallback: PowerShell SAPI.
Comunicacion	WebSocket (websockets >=14)	Bidireccional tiempo real. Eventos del juego en <5ms al browser.
Comunicacion browser	Web Serial API	Conexion directa browser <-> ESP32 sin software intermedio.
Web framework	Next.js 14 App Router	TypeScript nativo. Deploy Vercel. 100% client-side.
Animaciones web	Framer Motion	LEDs y transiciones sin CSS manual complejo.
ASR browser	@huggingface/transformers	Whisper WASM — fallback si no hay Python disponible.

Dependencias Python — pip install

Paquete	Version minima	Funcion
openai-whisper	latest	Reconocimiento de voz ASR local

pyserial	>=3.5	Comunicacion USB Serial con ESP32
websockets	>=14.0	Servidor WebSocket asyncio para el panel web
numpy	>=1.24	Arrays de audio PCM, operaciones vectorizadas
scipy	>=1.10	Filtros DSP (HPF/LPF, filtfilt) y resampling de audio
noisereduce	>=3.0	Sustraccion espectral de ruido ADC estacionario
sounddevice	>=0.4	Captura mic PC y reproduccion de tonos en modo simulador
pygame	>=2.0	Reproduccion de audio MP3 generado por edge-tts
edge-tts	>=6.0	Narrador TTS voz neural es-MX-DaliaNeural

Librerias Arduino — instalar desde Library Manager

Libreria	Version	Funcion
Adafruit SSD1306	>=2.5	Driver OLED SSD1306 128x32
Adafruit GFX Library	>=1.11	Base grafica requerida por SSD1306 (primitivas 2D)
Adafruit NeoPixel	>=1.12	Control LED RGB WS2812B en GPIO48
driver/i2s.h (nativa ESP-IDF)	incluida	I2S digital para speaker MAX98357A
Wire.h (nativa ESP-IDF)	incluida	I2C para OLED
math.h (C estandar)	incluida	sinf() para generar ondas sinusoidales en tonos I2S

12. Librerias Arduino — justificacion detallada

Wire.h — I2C

Que es: protocolo de comunicacion en bus de 2 hilos (SDA + SCL).

Por que: el OLED SSD1306 se comunica por I2C. Wire es la implementacion de Arduino.

Donde: Wire.begin(OLED_SDA, OLED_SCL) en setup(). Pines: SDA=GPIO10, SCL=GPIO11.

```
Wire.begin(10, 11);  // SDA=10, SCL=11
```

Adafruit_GFX.h — Motor grafico 2D

Que es: libreria base de Adafruit. Provee primitivas: texto, lineas, rectangulos, bitmaps.

Por que: es la dependencia requerida por Adafruit_SSD1306. No se usa directamente.

Donde: include implicito — Adafruit_SSD1306 la hereda.

Adafruit_SSD1306.h — Driver OLED

Que es: driver para pantalla OLED SSD1306 (128x32 px en este kit).

Por que: el kit tiene un OLED integrado que muestra estado del juego (nivel, puntos, mensajes).

Donde: oled_mostrar(l1, l2, l3) — imprime 3 lineas de texto.

```
display.clearDisplay();

display.setCursor(0, 0);

display.println('TU TURNO');

display.display();
```

Adafruit_NeoPixel.h — LED RGB WS2812B

Que es: libreria para LEDs NeoPixel (WS2812B). Control de color RGB por un pin de datos.

Por que: el ESP32-S3 del kit tiene un WS2812B integrado en GPIO48. Indica colores del juego y estado (grabando = rojo pulsante).

Donde: set_color(r, g, b) -> pixel.setPixelColor(0, ...) -> pixel.show().

```
pixel.setPixelColor(0, pixel.Color(255, 0, 0)); // rojo

pixel.show();
```

driver/i2s.h — I2S para speaker MAX98357A

Que es: driver de hardware ESP-IDF para protocolo I2S (transmision digital de audio).

Por que: el MAX98357A recibe audio digital I2S desde el ESP32. Usado para tonos de juego directamente en el speaker.

Donde: iniciar_i2s_speaker() en setup(), reproducir_tono_i2s(freq, ms) para tonos.

Pines: BCLK=GPIO6, WS/LRC=GPIO5, DIN=GPIO7.

NOTA: es la API de bajo nivel de Espressif (no libreria Arduino). Viene con el SDK de ESP32.

```
i2s_config_t cfg = {
    .mode = I2S_MODE_MASTER | I2S_MODE_TX,
    .sample_rate = 16000,
    .bits_per_sample = I2S_BITS_PER_SAMPLE_16BIT,
    .channel_format = I2S_CHANNEL_FMT_ONLY_LEFT,
};
```

math.h — Matematicas C estandar

Que es: libreria estandar de C para funciones matematicas.

Por que: necesaria para sinf() al generar ondas sinusoidales para los tonos I2S.

Donde: reproducir_tono_i2s() — genera muestras PCM de una onda seno.

```
buf[j] = (int16_t)(vol * 32767.0f * sinf(phase));
```

13. Librerias Python — justificacion detallada

openai-whisper

Que es: Implementacion oficial de OpenAI de Whisper ASR. Corre localmente, sin internet.

Por que: Reconocimiento de voz en espanol de alta calidad. Modelos tiny/base/small/medium. El proyecto usa small (~244MB).

Donde: whisper_engine.py: whisper.load_model('small') -> modelo.transcribe(audio_16k)

numpy

Que es: Libreria de Python para computacion numerica — arrays y operaciones vectorizadas.

Por que: El audio PCM llega como bytes crudos int16. Numpy convierte a float32 para filtros y metricas de SNR.

Donde: whisper_engine.py: np.frombuffer(pcm, dtype=np.int16).astype(float32) / 32768.0

scipy

Que es: Libreria cientifica. Usamos scipy.signal para DSP (filtros y resampling).

Por que: Implementa filtros IIR Butterworth con filtfilt (fase cero — sin distorsion temporal) y resample de alta calidad.

Donde: whisper_engine.py: butter() + filtfilt() para HPF/LPF | resample() para 8->16kHz

noisereduce

Que es: Libreria de sustraccion espectral de ruido estacionario.

Por que: El MAX4466 introduce mas ruido que el INMP441 digital. noisereduce estima el perfil de ruido del ADC y lo sustrae.

Donde: whisper_engine.py: nr.reduce_noise(y=audio, sr=8000, stationary=True, prop_decrease=0.55)

pyserial

Que es: Libreria para comunicacion por puerto Serial (UART/USB-CDC) en Python.

Por que: El ESP32 se conecta al PC por USB. Toda la comunicacion pasa por aqui — comandos de texto y bloques de audio PCM.

Donde: serial_bridge.py: serial.Serial(port, 921600) | deteccion automatica del puerto COM

websockets >=14

Que es: Implementacion Python del protocolo WebSocket RFC 6455.

Por que: El panel web necesita recibir eventos del juego en tiempo real (<5ms). La API asyncio.server solo existe desde v14.

Donde: ws_server.py: websockets.asyncio.server.serve(handler, 'localhost', 8766)

sounddevice

Que es: Binding de Python para PortAudio — reproduccion y captura de audio del sistema.

Por que: Captura el microfono del PC en modo simulador. Tambien reproduce tonos del juego en modo A.

Donde: audio_pc.py: sd.rec(n, samplerate=16000) | sd.play(muestras, 44100)

edge-tts

Que es: Cliente Python de Microsoft Edge TTS — voces neurales de Azure.

Por que: La voz es-MX-DaliaNeural es natural en espanol. Requiere internet solo para generar el audio MP3.

Donde: tts.py: edge_tts.Communicate(texto, 'es-MX-DaliaNeural').save('/tmp/tts.mp3')

pygame

Que es: Libreria multimedia de Python. Solo se usa pygame.mixer para reproduccion de audio MP3.

Por que: edge-tts genera MP3 temporales. pygame.mixer es la forma mas sencilla de reproducirlos en Python.

Donde: tts.py: pygame.mixer.music.load(mp3) | pygame.mixer.music.play()

Librerias estandar de Python (sin instalacion)

Libreria	Uso en el proyecto
threading	Hilos concurrentes: tick, TTS worker, secuencia, Whisper, serial reader
asyncio	Event loop del servidor WebSocket
time	Timeouts, sleeps, medicion de tiempos
json	Serializacion de mensajes WebSocket
queue	Cola thread-safe para TTS y WebSocket broadcast
unicodedata	Normalizacion de texto — quitar acentos en validador.py
re	Expresiones regulares para limpiar texto transcrito
difflib.SequenceMatcher	Fuzzy matching fonetico: 'adul' -> AZUL
collections.Counter	Deteccion de alucinaciones repetitivas de Whisper
random	Generacion de secuencias aleatorias de colores
enum	Definicion de estados del juego (Estado.LISTENING, etc.)
sys, os	Manipulacion de rutas y paths para imports entre modulos

14. Librerias Web Panel — justificacion detallada

next (14.2.3)

Que es: Framework React para produccion — App Router, SSR/SSG, builds optimizados.

Por que: Estructura moderna con TypeScript nativo, rutas basadas en archivos, deploy automatico en Vercel.

Donde: Toda la app en app/ — page.tsx es el dashboard. Usa 'use client' para componentes interactivos.

react + react-dom (18)

Que es: Libreria de UI declarativa basada en componentes y estado reactivo.

Por que: Base de Next.js. Los componentes del panel son React con hooks.

Donde: Hooks usados: useState, useEffect, useRef, useCallback en useWebSocket.ts y useWhisperWASM.ts.

@huggingface/transformers (3.5.0)

Que es: Ejecuta modelos de ML en el browser usando ONNX Runtime Web (WebAssembly + WebGPU).

Por que: Modo fallback WASM — si no hay Python, el browser transcribe con Whisper WASM (~125MB, cacheado en IndexedDB).

Donde: useWhisperWASM.ts: pipeline('automatic-speech-recognition', 'onnx-community/whisper-small', {dtype:'q4'})

framer-motion (12.x)

Que es: Libreria de animaciones para React — animaciones declarativas y transiciones.

Por que: Las transiciones del juego (LED activo, estado, resultado) se animan sin CSS manual complejo.

Donde: Componentes del dashboard — LEDPanel anima encendido/apagado, GameStatus hace fade in/out.

lucide-react (0.577)

Que es: Libreria de iconos SVG para React — ~1500 iconos como componentes.

Por que: Iconos del panel: microfono, conectar, nivel, puntuacion, log. SVG puro = escalable sin perder calidad.

Donde: Componentes UI: <Mic />, <Wifi />, <Trophy />, <Volume2 />, etc.

tailwindcss (3.3)

Que es: Framework CSS utility-first — clases atomicas directamente en JSX.

Por que: Desarrollo rapido de UI sin archivos .css separados. Output final solo incluye clases usadas (tree-shaking).

Donde: Todo el panel usa clases Tailwind: className='flex items-center gap-2 rounded-xl bg-zinc-900 p-4'

@radix-ui/react-scroll-area

Que es: Componente de area de scroll accesible (headless) de Radix UI.

Por que: El log en tiempo real necesita scroll suave con scrollbar personalizado (nativos no son estilizables).

Donde: LogConsole usa ScrollArea para el log de eventos con auto-scroll.

clsx + tailwind-merge

Que es: clsx: combina clases CSS condicionalmente. tailwind-merge: resuelve conflictos de clases Tailwind.

Por que: En componentes con props de className variable es facil tener conflictos. Juntos forman el patron cn().

Donde: Funcion cn() usada en casi todos los componentes: twMerge(clsx(inputs))

class-variance-authority (CVA)

Que es: Define variantes de componentes con type-safety — similar a variantes de Tailwind.

Por que: Botones, badges y chips del panel tienen multiples variantes tipadas (tamano, color, estado).

Donde: Componentes Button, Badge: define variantes 'default', 'destructive', 'outline', etc.

TypeScript (5.x)

Que es: Superset tipado de JavaScript — tipos estaticos comprobados en compilacion.

Por que: El protocolo WebSocket tiene 12 tipos de mensaje. Sin tipos, typos en msg.tipo pasan desapercibidos.

Donde: types/game.ts: MensajeWS como union discriminada con 12 tipos distintos.

15. Estructura de carpetas del proyecto

```
sistemas-inteligentes/
|
+-- firmware/
|   +-- test_hardware/
|   |   +-- test_hardware.ino     Menu interactivo: I2C, OLED, RGB, mic,
grabacion
|   |   +-- capturar_audio.py     Recibe PCM del ESP32 por Serial ->
guarda .wav
|   |   +-- test_whisper.py       Test completo: graba -> pipeline DSP ->
Whisper
|   +-- simon_dice/
|       +-- simon_dice.ino        Firmware del juego (actualmente: test
speaker MAX98357A)
|
+-- servidor_voz/
|   +-- main.py                   Punto de entrada: WebSocket :8766,
Whisper, PTT
|   +-- config_voz.py             Configuracion: modelo Whisper, puerto
serial, baud
|   +-- requirements.txt          Dependencias Python del servidor
|
+-- tests/
|   +-- simulador_pc/
|       +-- main.py               Arranca todos los hilos del simulador
|       +-- juego_sim.py          Maquina de estados del juego (Python)
|       +-- audio_pc.py           Mic PC + tonos + TTS en el simulador
|       +-- leds_sim.py           LEDs simulados en terminal ANSI
|       +-- ws_server.py          Servidor WebSocket :8765 para el panel
|       +-- validador.py          Texto -> comando canonico (AZUL, ROJO,
etc.)
|
+-- web-panel/
|   +-- app/
|   |   +-- page.tsx              Dashboard principal del juego
|   |   +-- layout.tsx            Shell HTML, metadata, fuentes
|   |   +-- globals.css           Estilos globales
|   |   +-- components/
|   |       +-- ConnectionPanel/  Boton conectar + selector modo + PTT +
info dispositivo
|   |       +-- GameStatus/       Badge estado + palabra detectada +
transcripcion
|   |       +-- LEDPanel/         4 LEDs virtuales animados de colores
|   |       +-- SequenceDisplay/  Secuencia visual del nivel actual como
cajas
|   |       +-- LogConsole/       Log en tiempo real con auto-scroll y
filtros
|   |       +-- ScoreBoard/       Nivel y puntuacion con animacion de
numeros
|   |       +-- TurnoTimer/       Cuenta regresiva durante estado
LISTENING
|   |       +-- SesionStats/      Mejor nivel, mejor puntuacion, total
partidas, racha
|   |       +-- HowToPlay/        Modal de ayuda con instrucciones y
```

```
vocabulario
|   +-- hooks/
|   |   +-- useWebSocket.ts       Conexion modo A/simulador (:8765)
|   |   +-- useWebSerial.ts       Conexion modo B/C (Web Serial API)
|   |   +-- useWhisperWASM.ts     Whisper WASM fallback en Web Worker
|   +-- types/
|   |   +-- game.ts               Tipos TypeScript del protocolo WebSocket
|   +-- lib/
|   |   +-- utils.ts              Funcion cn() para clases Tailwind
|   +-- package.json              Dependencias Node.js
|   +-- tailwind.config.ts        Configuracion Tailwind
|   +-- tsconfig.json             Configuracion TypeScript
|
+-- docs/
    +-- arquitectura_flujo.md     Diagramas Mermaid: flujo completo,
estados, WS
    +-- diagrama_arquitectura_completa.md  Tres modos, comparativa,
secuencias
    +-- diagrama_flujo_esp32.md   Flujo de partida, PTT fisico, OLED por
estado
    +-- diagrama_hardware_esp32.md   Hardware buses, pines del kit
    +-- librerias_y_stack.md      Justificacion detallada de cada libreria
    +-- narradora_frases.md       Todas las frases TTS y tonos musicales
    +-- setup.md                  Guia de instalacion paso a paso
    +-- generar_doc.py            Genera este documento .docx
```

16. Descripcion detallada de cada archivo

Firmware — firmware/

Archivo	Descripcion detallada
test_hardware/test_hardware.ino	Sketch de verificacion de hardware. Menu interactivo por Serial: (1) escaner I2C para localizar el OLED, (2) test OLED con texto de prueba, (3) test RGB con ciclo de colores, (4) VU meter del microfono MAX4466 en tiempo real, (5) grabacion de 10s y envio por Serial, (6) test Whisper con PTT. Pines verificados con este sketch: GPIO4=MAX4466, GPIO10=SDA, GPIO11=SCL, GPIO48=RGB, GPIO0=BOOT/PTT.
test_hardware/capturar_audio.py	Script Python para capturar el audio que el ESP32 envia por Serial y guardarlo como archivo .wav. Util para analizar la calidad del microfono MAX4466 antes de aplicar el pipeline DSP.
test_hardware/test_whisper.py	Test end-to-end del pipeline completo: activa grabacion en el ESP32, recibe PCM por Serial, aplica las 9 capas de DSP, transcribe con Whisper y muestra el resultado. Permite verificar la calidad del reconocimiento de voz con el hardware real.
simon_dice/simon_dice.ino	Firmware principal del juego. ESTADO ACTUAL: contiene el test del speaker MAX98357A (tono 1000Hz, 1 segundo). Pines: I2S_BCLK=GPIO6, I2S_WS=GPIO5, I2S_DOUT=GPIO7. El test verifica que el driver I2S funciona correctamente antes de integrar al juego completo.

Servidor de voz — servidor_voz/

Archivo	Descripcion detallada
main.py	Punto de entrada del servidor de voz. Abre el servidor WebSocket en el puerto 8766. Maneja dos fuentes de audio: (1) micrófono del PC via sounddevice cuando el usuario presiona ESPACIO en el browser, (2) audio base64 enviado por el ESP32 via USB Serial y reenviado por el browser via WebSocket. Transcribe con Whisper, valida el comando y lo envia de vuelta al browser y al ESP32.
config_voz.py	Configuracion centralizada: WHISPER_MODEL='small',

	SAMPLE_RATE=16000, WS_PORT=8766, SERIAL_BAUD=230400. Cambiar el modelo Whisper o el puerto serial desde aqui sin tocar main.py.
requirements.txt	Lista de dependencias Python para instalar con pip: openai-whisper, pyserial, websockets, numpy, scipy, noisereduce, sounddevice, pygame, edge-tts.

Simulador PC — tests/simulador_pc/

Archivo	Descripcion detallada
main.py	Entry point del simulador. Arranca todos los hilos en paralelo: tick (timer del juego), ws_server (WebSocket), audio_pc (microfono + TTS), secuencia (mostrar LEDs). Maneja la inicializacion y el apagado limpio de todos los hilos.
juego_sim.py	Maquina de estados completa del Simon Dice en Python. Contiene la clase Juego con estados (IDLE, SHOWING_SEQUENCE, LISTENING, EVALUATING, CORRECT, WRONG, LEVEL_UP, GAME_OVER, PAUSA). Genera secuencias aleatorias, evalua respuestas, gestiona el timer con pausa para TTS y Whisper, y llama callbacks on_estado, on_resultado, on_nivel, on_led.
audio_pc.py	Maneja el audio en el simulador: captura el microfono del PC con sounddevice cuando PTT esta activo, aplica el pipeline DSP (scipy + noisereduce + numpy), transcribe con Whisper, y reproduce tonos de juego (frecuencias de colores) y frases TTS (edge-tts).
leds_sim.py	Simula los LEDs del juego en la terminal usando codigos de color ANSI. Cuando el juego muestra ROJO, imprime un bloque de color rojo en la consola. Util para desarrollo sin hardware.
ws_server.py	Servidor WebSocket asyncio en el puerto 8765. Recibe eventos del juego (estado, LEDs, resultado) y los transmite al panel web en tiempo real como JSON. Tambien recibe comandos del panel (PTT, control) y los envia al juego.
validador.py	Convierte texto libre de Whisper a comandos canonicos del juego. Quita acentos

	(unicodedata), limpia puntuacion (re), busca coincidencias fuzzy (difflib.SequenceMatcher) para variantes foneticas ('adul'->'AZUL'). Expone texto_a_comando() y texto_a_colores() para modo multi-color.

Web Panel — web-panel/app/components/

Componente	Descripcion detallada
ConnectionPanel	Panel de conexion: selector de modo (WebSocket simulador / Web Serial ESP32), boton Conectar/Desconectar, indicador de estado de conexion, informacion del dispositivo Serial conectado. Muestra advertencia de Chrome/Edge si se detecta Firefox.
GameStatus	Badge del estado actual del juego (IDLE / MOSTRANDO / ESCUCHANDO / EVALUANDO / GAME OVER), ultima palabra detectada por Whisper, texto transcrito completo, indicador de TTS activo (narrador hablando).
LEDPanel	4 bloques de color (Rojo / Verde / Azul / Amarillo) con animacion Framer Motion. Se ilumina el color activo de la secuencia. En LISTENING los 4 LEDs pulsan levemente para indicar que escucha.
SequenceDisplay	Muestra la secuencia de colores del nivel actual como cajas compactas. Resalta en verde el ultimo color aceptado. Muestra la posicion actual del jugador en la secuencia.
LogConsole	Log en tiempo real de todos los eventos del juego. Auto-scroll al ultimo evento. Cada linea tiene timestamp, tipo de evento e icono de color. Scroll area con scrollbar personalizado (Radix UI).
ScoreBoard	Nivel actual y puntuacion con animacion de numeros al cambiar. Muestra el mejor nivel de la sesion.
TurnoTimer	Cuenta regresiva circular de 15s durante el estado LISTENING. Se pausa automaticamente cuando TTS esta hablando o Whisper esta procesando. Se reanuda donde quedo.
SesionStats	Estadisticas de la sesion actual: mejor nivel alcanzado, mejor puntuacion, total de partidas jugadas, racha maxima de niveles superados.

HowToPlay	Modal de ayuda con instrucciones del juego y lista completa de comandos de voz (colores, control, direcciones, respuestas).

Web Panel — web-panel/hooks/

Hook	Descripcion detallada
useWebSocket.ts	Conexion WebSocket al simulador PC (:8765). Maneja reconexion automatica, parseo de mensajes JSON tipados, envio de PTT (PTT_INICIO/PTT_FIN), y todas las actualizaciones de estado del juego al componente padre.
useWebSerial.ts	Conexion Web Serial API directamente al ESP32 (921600 baud). Lee lineas de texto del Serial, parsea el protocolo ESP32 (READY, STATE:, LED:, RESULT:, AUDIO:), y escribe comandos (ROJO, START, PTT_FIN, etc.). Maneja el flujo de audio base64 para enviarlo a servidor_voz.py.
useWhisperWASM.ts	Descarga Whisper WASM (~125MB, modelo small cuantizado INT4 de onnx-community) al IndexedDB del browser la primera vez. Transcribe audio Float32Array en un Web Worker para no bloquear la UI. Solo se activa si el servidor Python reporta whisperDisponible: false.

Web Panel — web-panel/types/

Archivo	Descripcion detallada
game.ts	Tipos TypeScript del protocolo WebSocket. Define MensajeWS como union discriminada de 12 tipos: ready, state, led, sequence, expected, voz, result, level, score, gameover, tts, log. Tambien define EstadoJuego (IDLE | SHOWING_SEQUENCE | LISTENING | ...) y ResultadoTurno (CORRECT | WRONG | TIMEOUT).

17. Firmware ESP32

El firmware corre en Arduino Framework sobre ESP-IDF. En Fase 1 solo gestiona hardware — la logica del juego vive en el servidor Python. El ESP32 captura audio, controla OLED y RGB, y se comunica por Serial.

Board settings en Arduino IDE

Setting	Valor
Board	ESP32S3 Dev Module
PSRAM	OPI PSRAM
Flash Size	16MB (128Mb)
Partition Scheme	Huge APP (3MB No OTA / 1MB SPIFFS)
USB Mode	Hardware CDC and JTAG
USB CDC on Boot	Enabled
Upload Speed	921600
CPU Frequency	240MHz (WiFi/BT)

Frecuencias de tonos por color (notas musicales)

Color	Frecuencia	Nota musical	Tono al mostrarse
ROJO	262 Hz	Do4 (C4)	Tono grave — fundamental
VERDE	330 Hz	Mi4 (E4)	Tono medio-bajo
AZUL	392 Hz	Sol4 (G4)	Tono medio
AMARILLO	494 Hz	Si4 (B4)	Tono agudo

Melodias de feedback

Evento	Secuencia de tonos
Inicio del juego	Do4 -> Mi4 -> Sol4 -> Do5 (262->330->392->523 Hz) — fanfarria ascendente
Respuesta correcta	Do6 -> Mi6 (1047->1319 Hz) — dos tonos agudos ascendentes
Respuesta incorrecta/timeout	Sol3 -> Mi3 (196->165 Hz) — dos tonos graves descendentes
Game Over	Do5 -> La4 -> Fa4 -> Do4 (523->440->349->262 Hz) — descendente dramatico

18. Servidor Python

El servidor Python (servidor_voz/main.py) no contiene logica del juego — esa esta en juego_sim.py (modo simulador) o en el firmware ESP32 (modo B/C). El servidor solo maneja: reconocimiento de voz, comunicacion Serial con ESP32, y comunicacion WebSocket con el panel.

Modos de captura de audio

Modo	Quien captura	Como llega al servidor
PTT teclado (barra espaciadora)	sounddevice (microfono del PC)	Browser envia PTT_INICIO / PTT_FIN por WebSocket
PTT fisico SW1 (boton ESP32)	ESP32 MAX4466 -> PSRAM -> base64	Browser reenvía audio base64 al servidor por WebSocket

Cadena de fallback TTS

```
1. edge-tts es-MX-DaliaNeural  (voz neural, requiere internet)

       |__ falla / sin internet

   2. PowerShell SAPI 'Microsoft Sabina Desktop'  (voz espanola del
sistema Windows)

           |__ sin voz espanola instalada

       3. PowerShell SAPI (voz en ingles del sistema)
```

Manejo de timeouts criticos en Whisper

El hilo de Whisper tiene un timeout de 10 segundos (configurable en config_voz.py). Si Whisper no termina en 10s, el servidor libera el hilo (_whisper_hilo_activo = None), envia DESCONOCIDO al panel para que limpie los flags de grabacion, y reanuda el timer del turno. El jugador puede intentar de nuevo inmediatamente. Esto evita que una transcripcion lenta bloquee el juego indefinidamente.

Validacion PTT — condiciones de rechazo

Condicion	Accion del servidor
TTS esta hablando (narradora activa)	Rechaza PTT, envia voz vacia al panel para limpiar flags. Log: 'Espera al narrador'
SHOWING_SEQUENCE activo (mostrando colores)	Rechaza PTT, envia voz vacia. Log: 'Mira la secuencia'
Whisper ya esta procesando otro audio	Rechaza PTT. El audio previo debe terminar primero.
Condiciones OK	Activa grabacion, envia 'R' al ESP32 por Serial

19. Panel Web — Next.js

El panel web corre 100% en el browser. No hay servidor backend en la nube. Deploy en Vercel: la conexion al ESP32 ocurre client-side desde el browser del usuario.

RESTRICCION IMPORTANTE: Web Serial API funciona SOLO en Chrome y Edge. Firefox y Safari no soportan esta API. El panel muestra una advertencia si detecta Firefox.

Resumen de componentes del dashboard

Componente	Descripcion
ConnectionPanel	Selector modo WebSocket/Serial, boton PTT, info del dispositivo conectado
LEDPanel	4 bloques de color (Rojo/Verde/Azul/Amarillo) — se iluminan segun el juego
GameStatus	Badge del estado actual, ultima palabra detectada, texto transcrito
SequenceDisplay	Secuencia de pasos del nivel actual como cajas de color compactas
LogConsole	Log en tiempo real de todos los eventos, auto-scroll, filtrable
ScoreBoard	Nivel y puntuacion con animacion de numeros
TurnoTimer	Cuenta regresiva de 15s durante el estado LISTENING, pausable
SesionStats	Mejor nivel, mejor puntuacion, total de partidas, racha maxima
HowToPlay	Modal de ayuda con instrucciones y lista de comandos de voz

Tipos TypeScript del protocolo (types/game.ts)

```
type MensajeWS =

  | { tipo: 'ready'; whisperDisponible: boolean; tiempoTimeout: number }

  | { tipo: 'state'; estado: EstadoJuego }

  | { tipo: 'led'; color: string | null }

  | { tipo: 'sequence'; secuencia: string[] }

  | { tipo: 'expected'; esperado: string }

  | { tipo: 'voz'; texto: string; comando: string }

  | { tipo: 'result'; resultado: 'CORRECT' | 'WRONG' | 'TIMEOUT' }

  | { tipo: 'level'; nivel: number }

  | { tipo: 'score'; puntuacion: number }
```

```
  | { tipo: 'gameover' }

  | { tipo: 'tts'; activo: boolean }

  | { tipo: 'log'; mensaje: string }
```

20. Vocabulario y validador de comandos

El vocabulario esta definido unicamente en vocabulario.h (firmware) y replicado en validador.py (Python) y validador.ts (TypeScript). Para agregar una palabra nueva: editar vocabulario.h y actualizar los validadores con la nueva variante fonetica.

Vocabulario completo

Categoria	Comandos
Colores	ROJO, VERDE, AZUL, AMARILLO
Control	START (o EMPIEZA, INICIA, COMIENZA), STOP, PAUSA, REPITE, REINICIAR
Direcciones	ARRIBA, ABAJO, IZQUIERDA, DERECHA
Respuestas	SI, NO
Especial	DESCONOCIDO (cuando no se reconoce nada)

Variantes foneticas reconocidas (fuzzy match)

Dicho por el jugador	Reconocido como	Razon
adul, asul, azul, a-zul	AZUL	Confusion Z/D en espanol regional
berde, berdi, verd	VERDE	Confusion B/V en espanol
roxo, roho, roca, roco	ROJO	Confusion J/X/H en espanol
amarilo, amarija, amariyo	AMARILLO	Simplificacion de 'LL'
empieza, inicia, comienza, arranca	START	Sinonimos de inicio
para, alto, detente, fin	STOP	Sinonimos de parada
bausa, pauca, pawsa	PAUSA	Confusion P/B y variantes foneticas
repita, repit, repitelo	REPITE	Variantes de imperativo
reinicia, renicia, reset	REINICIAR	Variantes y anglicismo

Modo multi-color — reglas de corte en texto_a_colores()

Cuando Whisper transcribe varios colores en un solo audio, el validador los procesa en orden. Se detiene en el primer token no reconocido.

Texto de Whisper	Resultado	Razon
'azul rojo rojo amarillo'	[AZUL, ROJO, ROJO, AMARILLO]	Todo reconocido
'adul dojo amadillo'	[AZUL, ROJO, AMARILLO]	Fuzzy matching activo
'azul rojo sdadsa verde'	[AZUL, ROJO]	Para en 'sdadsa' no reconocido
'sdadsa rojo rojo'	[]	Primera palabra falla -> camino normal
'reiniciar'	[]	No es color -> texto_a_comando ->

		REINICIAR
'azul' (solo 1)	[AZUL] -> camino normal	Menos de 2 colores usa camino estandar

21. Narradora TTS — frases y sonidos

Voz principal: es-MX-DaliaNeural (Edge TTS, requiere internet para generar el audio). Fallback offline: Microsoft Sabina Desktop (SAPI de Windows). El audio TTS pausa automaticamente el timer del turno para que el jugador no pierda tiempo mientras escucha.

Frases por evento del juego

Evento	Frase narrada
Arranque del servidor	Servidor listo. Abre el panel web y conecta.
Panel web conecta	Simon Dice listo. Presiona ESPACIO para comenzar.
Inicio de partida / SHOWING	Mira y escucha.
Durante la secuencia — colores	rojo / verde / azul / amarillo (uno por LED)
Turno del jugador (primer turno)	Tu turno. Presiona ESPACIO para hablar.
Turno del jugador (turnos siguientes)	Tu turno.
Acierto de un color (quedan mas)	Correcto. Tu turno.
Multi-color: 2 aceptados	2 colores correctos. Tu turno.
Multi-color: N aceptados	N colores correctos. Tu turno.
Secuencia completa — sube de nivel	Correcto. | Nivel N.
Color incorrecto — WRONG	Incorrecto. | Di empieza para intentar de nuevo.
Tiempo agotado — TIMEOUT	Tiempo agotado. | Di empieza para intentar de nuevo.
Pausa	Juego pausado.
Fin del juego — GAME OVER	Fin del juego. Obtuviste N puntos. | Di empieza para volver a jugar.

Tonos musicales por color (speaker MAX98357A)

Color o Evento	Tono (frecuencia)	Descripcion
ROJO en secuencia	Do4 — 262 Hz	Tono mas grave de los 4 colores
VERDE en secuencia	Mi4 — 330 Hz	Segundo tono
AZUL en secuencia	Sol4 — 392 Hz	Tercer tono
AMARILLO en secuencia	Si4 — 494 Hz	Tono mas agudo de los 4 colores
Respuesta correcta	Do6 -> Mi6 (1047->1319 Hz)	Dos tonos agudos ascendentes — exito
Respuesta incorrecta / timeout	Sol3 -> Mi3 (196->165 Hz)	Dos tonos graves descendentes — error
Inicio del juego	Do->Mi->Sol->Do (262->330->392->523 Hz)	Fanfarria ascendente de 4 notas
Game Over	Do->La->Fa->Do (523->440->349->262 Hz)	Melodia descendente dramatica

22. Pines GPIO verificados

Pines confirmados mediante pruebas fisicas con test_hardware.ino y observacion del PCB:

GPIO	Funcion	Modulo	Verificado	Notas
0	Boton BOOT / PTT (active LOW)	Boton integrado del kit	Si	INPUT_PULLUP, LOW al presionar
4	Microfono analogico OUT	MAX4466 externo	Si	ADC, 8kHz, oversample x4
5	I2S WS / LRC (speaker)	MAX98357A externo	Si	Word Select del I2S
6	I2S BCLK (speaker)	MAX98357A externo	Si	Bit Clock del I2S
7	I2S DIN (speaker)	MAX98357A externo	Si	Data In del I2S
10	I2C SDA (display)	OLED SSD1306 integrado	Si	Wire.begin(10, 11)
11	I2C SCL (display)	OLED SSD1306 integrado	Si	Wire.begin(10, 11)
35	Boton SW2	Boton integrado del kit	Pendiente	PTT alternativo
48	LED RGB data	WS2812B NeoPixel integrado	Si	1 pixel, Adafruit NeoPixel

NOTA: Los pines del INMP441 (microfono I2S digital integrado) aun no estan verificados. El proyecto usa MAX4466 (analogico) en GPIO4 como microfono activo. En Fase 2 se verificaran GPIO12/13/11 para el INMP441.

23. Hilos concurrentes del servidor

El servidor Python usa threading para manejar multiples operaciones en paralelo. Esto es critico porque Whisper (~3-8s) no puede bloquear el servidor WebSocket ni el timer del juego.

Hilo	Nombre interno	Responsabilidad
Principal	main	Arranque, configuracion, bucle while True
WS server	ws-server	asyncio event loop — recibe y envia mensajes WebSocket
Serial reader	serial-reader	Lee stream Serial del ESP32 linea por linea
Audio proc	audio-proc (spawneado)	Recibe PCM del Serial, ejecuta pipeline DSP y llama Whisper
Whisper	whisper-transcribe	Transcripcion con timeout de 10s maximo
TTS worker	tts-worker	Reproduce cola TTS en orden sin bloquear el juego
Tick	tick	Verifica timeout del turno cada 200ms. Detecta cambios de TTS para notificar al panel.
Secuencia	seq	Muestra LEDs de la secuencia (bloqueante — espera LED+pausa por color)
Nivel up	nivel-up	Delay de 0.6s antes de iniciar el nuevo nivel
Game over	gameover	Delay de 0.8s antes de entrar en estado GAME_OVER
Bienvenida	bienvenida	Reproduce TTS de bienvenida al conectar el primer cliente WebSocket

Sincronizacion critica: el hilo 'tick' pausa el timer cuando _tts_activo.is_set() o _whisper_procesando es True. Esto garantiza que el jugador no pierde tiempo mientras la narradora habla o Whisper esta transcribiendo.

24. Display OLED — mensajes por estado

El OLED SSD1306 de 128x32 pixeles muestra 3 lineas de texto de 8x8 pixeles cada una. El contenido cambia en cada transicion de estado del juego.

Estado del juego	Linea 1 (OLED)	Linea 2 (OLED)	Linea 3 (OLED)
IDLE	Simon Dice	Di EMPIEZA	para comenzar
SHOWING_SEQUENCE	MOSTRANDO	secuencia	(en blanco)
LISTENING	TU TURNO	Nv2 Pts:10 1/2	Presiona ESPACIO
EVALUATING	Procesando...	Whisper	(en blanco)
CORRECT	CORRECTO!	(en blanco)	(en blanco)
LEVEL_UP	NIVEL 3!	Puntos: 20	Bien hecho!
WRONG	INCORRECTO	(en blanco)	(en blanco)
GAME_OVER	GAME OVER	(en blanco)	Di EMPIEZA
PAUSA	PAUSA	Di EMPIEZA	para continuar

Nota: el estado LISTENING es dinamico — muestra nivel y puntuacion actuales y la posicion en la secuencia (1/2, 2/3, etc.).

25. Setup e instalacion

Requisitos previos

- Arduino IDE >= 2.0 con soporte ESP32 instalado (URL: espressif.github.io/arduino-esp32/package_esp32_index.json)
- Python >= 3.10
- Node.js >= 18
- Chrome o Edge (para el panel web — Firefox no soporta Web Serial API)
- Cable USB-C
- ~5GB de espacio libre (modelos Whisper + dependencias Node/Python)

1. Firmware ESP32 — instalar y flashear

```
# 1. Instalar librerias en Arduino IDE -> Tools -> Manage Libraries:

#    Adafruit SSD1306  |  Adafruit GFX Library  |  Adafruit NeoPixel

#

# 2. Configurar board settings:

#    Board: ESP32S3 Dev Module

#    PSRAM: OPI PSRAM | Flash: 16MB | Partition: Huge APP

#    USB Mode: Hardware CDC and JTAG | USB CDC on Boot: Enabled

#

# 3. Abrir firmware/test_hardware/test_hardware.ino

# 4. Upload (921600 baud)

# 5. Abrir Serial Monitor a 921600 baud y verificar hardware
```

2. Servidor de voz

```
cd servidor_voz

pip install -r requirements.txt

# El modelo Whisper small (~244MB) se descarga automaticamente la primera
vez

python main.py

# Output esperado: 'Servidor listo en ws://localhost:8766'
```

3. Simulador PC (sin hardware ESP32)

```
cd tests/simulador_pc

pip install -r requirements_test.txt

python main.py

# Output esperado: 'Simon Dice Simulador iniciado | WebSocket :8765'
```

4. Panel Web

```
cd web-panel

npm install

npm run dev

# Abrir http://localhost:3000 en Chrome o Edge

# En Firefox: aparecera advertencia de compatibilidad
```

5. Orden de inicio (Modo B — ESP32 + servidor)

10. Flashear firmware al ESP32 (test_hardware.ino o simon_dice.ino)
11. Conectar ESP32 por USB al PC
12. python servidor_voz/main.py (primero el servidor — se conecta al puerto Serial automaticamente)
13. Abrir Chrome -> http://localhost:3000
14. Click 'Conectar al ESP32' en el panel
15. Verificar que el log muestra 'READY' del ESP32
16. Decir EMPIEZA para iniciar la primera partida

6. Orden de inicio (Modo A — Simulador sin hardware)

17. python tests/simulador_pc/main.py
18. Abrir Chrome -> http://localhost:3000
19. Click 'Conectar al Simulador' (WebSocket :8765)
20. Decir EMPIEZA (barra espaciadora + hablar) para iniciar

26. Riesgos y decisiones de diseno

Tabla de riesgos

Riesgo	Impacto	Mitigacion
Pines GPIO sin esquematico oficial del kit	Alto	Semana de pruebas con test_hardware.ino — pines ya verificados fisicamente
Whisper lento en CPU debil (>8s)	Medio	Fallback a modelo tiny en config_voz.py. Timeout de 10s libera el hilo.
Ruido ADC del MAX4466	Alto	Pipeline de 9 capas: HPF+LPF firmware, HPF+LPF Python, noisereduce, normalizacion, resample
Web Serial API solo en Chrome/Edge	Medio	Documentado en UI. Modo WebSocket funciona en cualquier browser.
Tiempo insuficiente para Fase 2	Bajo	Fase 1 (con PC) = entrega minima viable completa y funcional
Alucinaciones de Whisper	Medio	difflib fuzzy match + collections.Counter detecta patrones repetitivos anormales
Timeout Whisper bloquea el juego	Medio	Hilo separado con join(timeout=10s). Si expira, libera y reanuda timer.
edge-tts sin internet	Bajo	Fallback automatico a PowerShell SAPI (voz del sistema Windows)

Decisiones de diseno clave

Por que MAX4466 y no INMP441?

El kit tiene INMP441 (I2S digital), pero las pruebas iniciales se hicieron con MAX4466 (analogico) porque el pinout del INMP441 no estaba verificado. El MAX4466 funciona bien con el pipeline DSP compensando su mayor ruido. En Fase 2 se migrara al INMP441 para mejor calidad sin DSP.

Por que texto plano en el protocolo Serial y no JSON?

El texto plano es legible directamente en el Serial Monitor del Arduino IDE sin herramientas adicionales. Facilita enormemente el debugging durante el desarrollo. JSON se usa solo en WebSocket (Python <-> Browser) donde hay parsers nativos.

Por que WebSocket y no HTTP polling?

Los eventos del juego deben llegar al browser en tiempo real (<50ms). HTTP polling introduce latencia inaceptable para la experiencia de juego. WebSocket mantiene la conexion abierta y envia eventos inmediatamente cuando ocurren.

Por que un servidor Python separado y no todo en el browser?

Whisper Python small (244MB float32) tiene mejor calidad que Whisper WASM (125MB INT4) porque usa precision completa y aprovecha optimizaciones de CPU (AVX2). El servidor Python es la opcion de mayor calidad; el WASM es el fallback cuando no hay Python disponible.

Por que el vocabulario solo en vocabulario.h?

Para que agregar una palabra nueva requiera cambiar un solo archivo. Si el vocabulario estuviera duplicado en firmware, servidor Python, validador y panel, cada cambio requeriria 4 ediciones sincronizadas — fuente segura de bugs en un proyecto universitario con tiempo limitado.

Por que timer pausado durante TTS y Whisper?

Si el timer corriera mientras la narradora habla, el jugador perderia segundos valiosos simplemente escuchando 'Correcto. Tu turno.' El jugador solo debe contar tiempo cuando el sistema esta activamente esperando su respuesta, no cuando el sistema esta ocupado.

Por que Next.js y no React puro o Vue?

Next.js 14 con App Router ofrece deploy automatico en Vercel con cero configuracion, TypeScript nativo, y el panel es completamente client-side (sin backend en la nube). El panel corre en el browser del usuario — la conexion al ESP32 ocurre localmente.